Zonoth

SESSION 05 · 2H

# Kubernetes: architecture, kubectl, Pods

From one machine running Compose to a cluster that keeps Pinboard running on its own — and the one command you drive it with.

Why orchestration | Cluster architecture | API & kubectl | Pods in depth | Lab 05

SESSION 05

# What we'll cover today

## 1 · Cluster architecture

Why orchestration, what Kubernetes is and isn't, and every process on the control plane and the nodes.

## 2 · The API & kubectl

The object model, desired state and reconciliation, kubectl grammar, kubeconfig, contexts and namespaces.

## 3 · Pods in depth

Shared network and volumes, lifecycle and status, requests/limits and QoS, init and sidecar containers.

## 4 · Lab 05

Create the kind cluster, load `pinboard-api:1.0` and run it as a Pod in the `pinboard` namespace.

01

# Why Kubernetes

Compose gave us one host. Production needs many — and something that watches them for us.

MOTIVATION

# What Compose could not do

## Many hosts

Compose schedules onto **this** machine. Nothing spreads Pinboard over a fleet.

## Self-healing

If the host dies, the containers die with it. Nobody restarts them elsewhere.

## Rolling updates

`up -d` replaces containers all at once — a visible gap in service.

## Scaling

`--scale api=3` is manual and capped by one machine's CPU.

## Secrets & config

A `.env` file on a laptop is not a secret management story.

## Traffic routing

No load balancing, no service discovery beyond a single bridge network.

DEFINITION

# What Kubernetes is — and isn't

## It is

- A **container orchestrator**: it places containers on machines and keeps them there
- A **declarative API** you POST desired state to
- A set of **controllers** that reconcile reality toward that state
- Extensible — CRDs, operators, CNI/CSI/CRI plugins

## It is not

- A PaaS — it does not build your code or run a CI pipeline
- A container runtime — that is containerd, via the CRI
- A database, a message bus or a logging stack
- Free of operations — it moves the work, it does not delete it

Kubernetes 1.37 (kind v0.33) is the baseline for this course. The project ships three minor releases a year and supports the last three.

WHERE IT RUNS

# Local clusters vs managed clusters

| LOCAL — FOR LEARNING & CI | MANAGED — FOR PRODUCTION |
| --- | --- |
| **kind — nodes are Docker containers; multi-node, fast, disposable.** Our choice. | **AKS · EKS · GKE** — the cloud runs and upgrades the control plane for you. |
| **minikube — VM or Docker driver, rich addons** | You still own nodes, upgrades, quotas and cost |
| **k3d (k3s in Docker) · Docker Desktop single-node** | Real LoadBalancers, real StorageClasses, cloud IAM |

✓ The API is identical everywhere. What changes is what a `LoadBalancer` Service or a `PersistentVolumeClaim` resolves to.

02

# Cluster architecture

A control plane that decides, and nodes that do. Nine processes, and you only ever talk to one of them.

THE BRAIN

# Control plane components

## kube-apiserver

The only door into the cluster. REST over HTTPS, authn/authz, admission, validation.

## etcd

Consistent key-value store — the single source of truth. Back it up to rebuild.

## kube-scheduler

Watches for Pods with no node, scores candidates, writes the binding back to the API.

## kube-controller-manager

Dozens of control loops in one binary — Deployment, Node, Job…

## cloud-controller-manager

Cloud-specific loops: LoadBalancers, routes, node lifecycle. Not on kind.

## CoreDNS (add-on)

Cluster DNS. Not strictly control plane, but every cluster has it — Session 07.

THE MUSCLE

# Node components

## kubelet

The node agent. Watches the API for Pods bound to its node, asks the runtime to start them, runs probes, reports status back.

## kube-proxy

Programs iptables/IPVS (or is replaced by an eBPF CNI) so Service IPs reach Pod IPs.

## Container runtime

**containerd** via the **CRI**, using runc and OCI images — the same images you built in Session 02.

⚠ **Dockershim was removed in Kubernetes 1.24.** "Kubernetes dropped Docker" only ever meant the shim — your Docker-built OCI images run unchanged on containerd.

FOLLOW THE REQUEST

# What happens when you apply a Pod

**01 — kubectl apply**
YAML → JSON, POST to the apiserver

→

**02 — apiserver**
Authn, authz, admission, validate

→

**03 — etcd**
Object persisted, `nodeName` empty

→

**04 — scheduler**
Picks a node, writes the binding

→

**05 — kubelet**
Pulls image, CRI starts it, reports Running

Every arrow is a **watch** on the API, not a direct call. No component ever phones another — they all react to the same store.

LIVE DEMO · 8 MIN

# Build a cluster and look inside it

Create the three-node course cluster with kind, then prove that the control plane is itself just Pods running on Kubernetes.

**What you'll see** · nodes, control-plane Pods, containerd as the runtime **Files** · labs/kind-config.yaml **Duration** · 8 min

03

# The API and kubectl

Every object has the same four fields. Every command has the same three words.

# Four fields, every single time

- **apiVersion** — group and version, e.g. `v1`, `apps/v1`
- **kind** — the type: Pod, Deployment, Service
- **metadata** — name, namespace, labels, annotations
- **spec** — **what you want**; you write this
- **status** — **what is**; the cluster writes this

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pinboard-api
  namespace: pinboard
  labels:
    app.kubernetes.io/name: pinboard-api
spec:
  containers:
    - name: api
      image: pinboard-api:1.0
      imagePullPolicy: IfNotPresent
      ports:
        - { name: http, containerPort: 8080 }
```

THE CORE IDEA

# Desired state and the reconciliation loop

**01 — Declare**
You write `spec` and apply it

→

**02 — Observe**
Controller watches actual state

→

**03 — Diff**
Compare desired vs actual

→

**04 — Act**
Create, delete or update to close the gap

↻

**05 — Repeat**
Forever — this is self-healing

Same idea as `terraform apply` — except nobody has to run it. The loop never stops, so drift is corrected in seconds.

ONE COMMAND TO LEARN

# kubectl grammar

BASH

```
kubectl [verb] [resource] [name] [flags]

kubectl get pods                          # list
kubectl get pod pinboard-api -o yaml      # one object, full spec+status
kubectl describe pod pinboard-api         # human summary + events
kubectl logs pinboard-api -c api -f       # container logs, follow
kubectl exec -it pinboard-api -- sh       # a shell inside the container
kubectl port-forward pod/pinboard-api 8080:8080
kubectl delete pod pinboard-api
kubectl get pods -o wide --show-labels    # node, IP, labels
kubectl get pods -l app.kubernetes.io/name=pinboard-api
```

Verbs: `get describe create apply delete edit logs exec scale rollout`. Resources take short names (`po`, `deploy`, `svc`, `ns`) and are always plural-friendly.

TWO STYLES

# Imperative vs declarative

## Imperative — fast, unrepeatable

BASH

```
kubectl run pinboard-api \
  --image=pinboard-api:1.0 \
  -n pinboard --port=8080
```

Great for a five-second experiment. Nothing is written down; nobody can review it.

## Declarative — the real workflow

BASH

```
kubectl apply -f pod.yaml
kubectl diff -f pod.yaml
kubectl apply -f k8s/ --dry-run=server
```

The file is the source of truth: reviewable, versioned, re-appliable, and the input to GitOps.

Best of both: `kubectl create deployment … --dry-run=client -o yaml > deploy.yaml` — generate the skeleton, then edit and commit it.

# kubeconfig and contexts

- `~/.kube/config` — clusters, users, and **contexts** that pair them
- A context also carries a **default namespace**
- `$KUBECONFIG` can merge several files with `:`
- `kind create cluster` adds and selects `kind-pinboard`

> ⚠ Every production incident story starts with "I thought I was on dev." Put the context in your shell prompt — kube-ps1, starship, k9s.

BASH

```
kubectl config get-contexts
kubectl config current-context
# kind-pinboard

kubectl config use-context kind-pinboard

# pin the namespace for this context
kubectl config set-context --current \
  --namespace=pinboard

kubectl config view --minify
```

SCOPING

# Namespaces

- A **name scope**, not a security boundary by itself
- Names must be unique **per namespace**; DNS becomes `svc.ns.svc.cluster.local`
- The hook for ResourceQuota, LimitRange, RBAC and NetworkPolicy
- Cluster-scoped objects have none: Node, PersistentVolume, StorageClass, Namespace

BASH

```
kubectl create namespace pinboard
kubectl get ns
# default   kube-system   kube-public
# kube-node-lease   pinboard

kubectl get pods -n pinboard
kubectl get pods -A          # all namespaces

kubectl api-resources --namespaced=false
```

✓ Ours is `pinboard` for every lab from here on.

SELF-SERVICE DOCS

# kubectl explain — the API reference offline

BASH

```
kubectl explain pod.spec
kubectl explain pod.spec.containers.resources
kubectl explain deployment.spec.strategy --recursive

kubectl api-resources | head
# NAME          SHORTNAMES   APIVERSION   NAMESPACED   KIND
# pods          po           v1           true         Pod
# deployments   deploy       apps/v1      true         Deployment

kubectl api-versions | grep apps
```

`explain` reads the OpenAPI schema from **your** cluster, so it is always right for the version you are on — including CRDs your team installed.

LIVE DEMO · 12 MIN

# A Pod from YAML, watched end to end

Apply the manifest, watch the Pod fail to pull `pinboard-api:1.0`, side-load it with `kind load` — and watch the events narrate it recovering on its own.

**What you'll see** · explain · apply · ErrImagePull · kind load · port-forward

**Files** · demos/session05/namespace.yaml · pod.yaml

**Duration** · 12 min

04

# Pods in depth

The smallest thing Kubernetes will schedule — and the only thing that ever runs your containers.

ANATOMY

# What a Pod actually shares

- **One network namespace** — one IP, one port space; containers reach each other on `localhost`
- **Volumes** — declared on the Pod, mounted per container
- **Lifecycle & node** — scheduled together, killed together
- **Not** shared: filesystem root, PID namespace (opt-in), memory limits

⚠ Two containers cannot both bind `:8080` in one Pod — same port space.



## One app per Pod

`pinboard-api` and `pinboard-web` are **separate** Pods: they scale differently, fail differently and are released differently.

Extra containers belong in a Pod only when they are useless alone — a log shipper, a proxy, a config reloader.

READING THE ROOM

# Pod lifecycle and the status you will see

| STATUS | WHAT IT MEANS | FIRST MOVE |
|---|---|---|
| **Pending** | Accepted, not yet bound — no node fits, or a quota/taint blocks it | `describe pod` → Events |
| **ContainerCreating** | Node chosen; pulling the image, mounting volumes, wiring the network | Wait, then `describe` |
| **Running** | At least one container is up. **Ready** is a different column | `logs`, check READY `1/1` |
| **CrashLoopBackOff** | The container starts and exits repeatedly; kubelet backs off up to 5 min | `logs --previous` |
| **ImagePullBackOff** | Image missing, tag wrong, or registry auth failed | Check tag; `kind load` |
| **Completed / Error** | Process exited 0 or non-zero and `restartPolicy` allowed it to stop | Expected for Jobs |

# Requests, limits and QoS

- **requests** — what the **scheduler** reserves
- **limits** — what the **kubelet** enforces
- Over CPU → throttled. Over memory → **OOMKilled**

| QOS CLASS | WHEN |
|---|---|
| **Guaranteed** | requests == limits, every container |
| **Burstable** | requests set, limits differ or absent |
| **BestEffort** | neither set — evicted first |

YAML

```yaml
resources:
  requests:
    cpu: 50m          # 0.05 of a core
    memory: 32Mi
  limits:
    memory: 64Mi      # no cpu limit on purpose
```

Common production stance: **always** set memory request = limit, and leave CPU unlimited so bursts are absorbed instead of throttled.

# Init containers and native sidecars

- **initContainers** run to completion, in order, before app containers start
- A **native sidecar** is an init container with `restartPolicy: Always` — on by default since 1.29, GA in 1.33
- Sidecars start **before** the app, keep running, and stop **after** it
- That ordering is what the old "sidecar as a normal container" hack never got right

YAML

```yaml
spec:
  initContainers:
    - name: wait-for-db
      image: busybox:1.37
      command: ["sh","-c",
        "until nc -z pinboard-db 5432; do sleep 1; done"]
    - name: log-shipper
      image: busybox:1.37
      restartPolicy: Always     # ← native sidecar
      command: ["sh","-c","tail -f /var/log/app.log"]
  containers:
    - name: api
      image: pinboard-api:1.0
```

DAY-TO-DAY

# restartPolicy and working with a live Pod

- `restartPolicy: Always` (default) · `OnFailure` · `Never`
- It restarts **containers in place** — the Pod is never rescheduled
- A bare Pod on a dead node is simply gone. Nothing recreates it
- That is why real workloads are owned by a controller — Session 06

> ⚠ Never run a bare Pod in production. It is a debugging tool.

BASH

```
kubectl describe pod pinboard-api -n pinboard
kubectl logs pinboard-api -n pinboard -f
kubectl logs pinboard-api -n pinboard --previous
kubectl exec -it pinboard-api -n pinboard -- sh
kubectl port-forward pod/pinboard-api 8080:8080 \
  -n pinboard
curl -s localhost:8080/api/info
```

LAB 05 · 30–40 MIN

# Pinboard meets Kubernetes

Create the three-node kind cluster, side-load the image you built in Lab 02, and run `pinboard-api:1.0` as a Pod in the `pinboard` namespace — first imperatively, then from a manifest you keep.

**You'll practise:** `kind create cluster` · `kind load docker-image` · `kubectl run` vs `apply` · namespaces & contexts · `port-forward` · `logs` · `describe` · requests/limits · reading an ImagePullBackOff · `kubectl explain`

# Key takeaways

## ✓ One door, many loops

Everything goes through kube-apiserver into etcd; scheduler, controllers and kubelet just watch and react.

## ✓ spec is yours, status is theirs

Declare desired state and the reconciliation loop closes the gap — continuously, without you re-running anything.

## ✓ The Pod is the unit

Shared IP and volumes, requests/limits for scheduling, init and native sidecar containers for ordering.

## ✓ Next up

Session 06 — Deployments, rolling updates and the probes that make them safe.

Zonoth

# Questions?

Up next — Session 06: Workloads, rollouts and probes. Leave the kind cluster running.